Miniexamen 8

Curso de Python

Nombre: _____

Guía para aprender matplotlib con el archivo `gfs_data.nc`. Hagan cada paso en orden y guarden su código y sus gráficas.

1. Abrir el archivo

1. Abran el archivo e impriman el Dataset:

```
from pathlib import Path
import numpy as np
import xarray as xr
import matplotlib.pyplot as plt

file = Path.home() / 'Downloads' / 'gfs_data.nc'
ds = xr.open_dataset(file)
print(ds)
```

2. ¿Cuántas variables trae? Imprimánlas todas con `print(ds.data_vars)`.
3. Impriman las dimensiones de `t` y de `zh` (`ds['t'].dims`). ¿Son las mismas?

2. La temperatura a lo largo de la latitud

`t` tiene tres dimensiones: `(lev, lat, lon)`. Si fijamos dos, queda una línea que podemos graficar. Aquí fijamos una longitud y un nivel.

Los niveles se cuentan **de arriba hacia abajo**: `lev=0` es el tope del modelo.

1. Saquen la temperatura en la columna `lon=100`, en los niveles 2 y 10. `sel` selecciona por el valor de la coordenada; aquí `lev` y `lon` ya vienen numerados desde 0:

```
t_2  = ds['t'].sel(lon=100, lev=2)
t_10 = ds['t'].sel(lon=100, lev=10)
lat  = t_10['geolat']     # la latitud de cada punto

print(t_10.dims)          # solo queda lat
```

2. ¿A qué altura está el nivel 10? La altura `zh` está en las fronteras entre niveles (`levp`): el nivel `k` queda entre `levp=k` y `levp=k+1`.

```
z = ds['zh'].sel(lon=100, levp=[10, 11]).mean('lat')
print(z.values)
```

   Revisen las unidades en `ds['zh'].attrs`. ¿Y el nivel 2?
3. No usen `lev=0` para alturas: su frontera de arriba da un valor absurdo.

3. Dos formas de graficar

Figure es toda la hoja. **Axes** es una gráfica dentro de la hoja, con sus ejes, título, leyenda y líneas. Una Figure puede tener varios Axes.

Con `plt`

```
plt.plot(lat, t_10)
plt.title('Temperatura, nivel 10')
plt.xlabel('Latitud [°]')
plt.ylabel('Temperatura [K]')
plt.show()
```

Con objetos

```
fig, ax = plt.subplots()
ax.plot(lat, t_10)
ax.set_title('Temperatura, nivel 10')
ax.set_xlabel('Latitud [°]')
ax.set_ylabel('Temperatura [K]')
plt.show()
```

- `plt.subplots()` crea la hoja y una gráfica, y las guarda en dos variables: `fig` (la Figure) y `ax` (el Axes).
- Con `plt`, matplotlib dibuja en "la gráfica actual", la última que se creó. Tú no eliges cuál.
- Con objetos, tú dices en qué gráfica dibujar: `ax.plot(...)`. Con `plt` también se pueden hacer varias gráficas (`plt.subplot`), pero hay que ir cambiando de "gráfica actual"; con objetos le hablas directo a cada una.
- Los nombres cambian: `plt.title` → `ax.set_title`, `plt.xlabel` → `ax.set_xlabel`.

1. Hagan las dos versiones. ¿Salen iguales?
2. De aquí en adelante, usen siempre la versión con objetos.

4. La línea y los marcadores

Se cambian dentro de `ax.plot(lat, t_10, ...)`. Agreguen una propiedad a la vez y vean qué cambia.

Propiedad	Prueben
`color`	`'crimson'`, `'steelblue'`, `'#4ecdc4'`
`linestyle`	`'-'`, `'--'`, `':'`, `'-.'`, `'none'`, `(0, (5, 10))`
`linewidth`	`0.5`, `2`, `4`
`marker`	`'o'`, `'s'`, `'^'`, `'D'`, `'x'`
`markersize`	`3`, `8`, `12`
`markevery`	`10` (un marcador cada 10 puntos)
`markerfacecolor`	`'white'`, `'yellow'`
`markeredgecolor`	`'black'`, `'blue'`
`markeredgewidth`	`0.5`, `2`
`fillstyle`	`'full'`, `'left'`, `'bottom'`, `'none'`
`alpha`	`0.3`, `0.5`, `1.0`
`label`	`'nivel 10'` (texto para la leyenda)

1. Prueben cada propiedad de la tabla.
2. Pongan `linestyle='none'` con un marcador. ¿Qué queda?
3. Usen marcadores de texto: `r'$\heartsuit$'`, `r'$\alpha$'`. Para ver cuáles hay:

```
from matplotlib._mathtext_data import tex2uni
print(sorted(tex2uni))
```

4. `fillstyle` solo funciona con marcadores rellenables como `'o'` o `'s'`. Compruébenlo con uno de texto.
5. Forma corta: `ax.plot(lat, t_10, 'r--o')` es línea roja, discontinua, con círculos. Prueben otras combinaciones.
6. *(Opcional)* Vean los estilos con `plt.style.available` y prueben dos con `plt.style.use(...)`.

5. La gráfica: títulos, ejes, leyenda, guardar

Método	Qué hace
`plt.subplots(figsize=(8, 4))`	tamaño de la hoja, en pulgadas
`ax.set_title('...', fontsize=14)`	título y su tamaño
`ax.set_xlabel`, `ax.set_ylabel`	nombre de cada eje
`ax.legend(loc='upper left')`	leyenda (usa los `label`); `loc` dice dónde
`ax.grid(True, linestyle='--', alpha=0.3)`	cuadrícula
`ax.set_xlim(0, 40)`, `ax.set_ylim`	rango de cada eje
`fig.tight_layout()`	acomoda para que nada se encime
`fig.savefig('t10.png', dpi=150)`	guarda la figura

Dónde poner la leyenda. `loc` recibe un texto con la posición dentro de la gráfica:

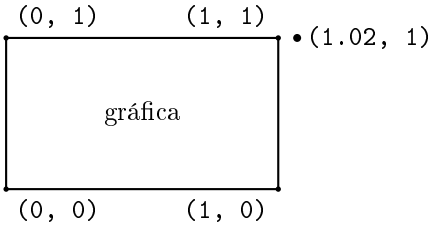
	izquierda	centro	derecha
arriba	`'upper left'`	`'upper center'`	`'upper right'`
en medio	`'center left'`	`'center'`	`'center right'`
abajo	`'lower left'`	`'lower center'`	`'lower right'`

Además: `'best'` (la que usa si no pones nada: busca donde menos tape las líneas) y `'right'` (igual que `'center right'`).

Para sacarla de la gráfica, se combina con `bbox_to_anchor`:

```
ax.legend(loc='upper left', bbox_to_anchor=(1.02, 1))   # afuera, a la derecha
```

`(1.02, 1)` está en **fracciones de la gráfica**: se miden sobre el rectángulo de los ejes, no sobre toda la hoja.



- `(0, 0)`: esquina inferior izquierda; `(1, 1)`: superior derecha; `(0.5, 0.5)`: el centro.
- Mayor que 1 o menor que 0: **fuera** de la gráfica.

Así, `(1.02, 1)` es un punto pegado al borde derecho, a la altura del borde de arriba. Con `loc='upper left'`, la esquina superior izquierda de la leyenda se pega a ese punto y la leyenda queda afuera, colgando hacia abajo. Son las mismas coordenadas que `transform=ax.transAxes` en la sección 12.

1. Pongan `t_2` y `t_10` en los mismos ejes, cada una con su `label`, y muestren la leyenda.
2. Muévanla a tres posiciones de la tabla con `loc=`. Luego prueben `loc='best'`.
3. Sáquenla de la gráfica con `bbox_to_anchor`. Cambien el `loc` y vean qué esquina se pega al punto.
4. Agreguen título y etiquetas con unidades.
5. Limiten el eje x a latitudes entre 0 y 40.
6. Agreguen la cuadrícula y prueben `alpha` y `linestyle` en ella.
7. Cambien `figsize` y guarden la figura en PNG (`dpi=150`) y en PDF. Comparén con zoom.

6. Varias gráficas en una hoja

a) Con `plt.subplot` (en singular): la forma con `plt`.

```
plt.figure(figsize=(8, 6))
plt.subplot(2, 1, 1)      # 2 filas, 1 columna, gráfica 1
plt.plot(lat, t_2)
plt.title('Nivel 2')
plt.subplot(2, 1, 2)      # gráfica 2
plt.plot(lat, t_10)
plt.title('Nivel 10')
plt.tight_layout()
plt.show()
```

1. Hagan esta versión. Las gráficas se numeran desde 1, por filas.
2. Después de dibujar las dos, agréguenle una cuadrícula solo a la primera. ¿Qué tienen que hacer?

b) Con `plt.subplots` (en plural): la forma con objetos.

```
fig, axs = plt.subplots(2, 1, figsize=(8, 6))
axs[0].plot(lat, t_2)
axs[1].plot(lat, t_10)
```

1. `axs` es un arreglo de Axes. Pónganle título y etiquetas a cada uno.
2. Repítanlo desempacando: `fig, (ax1, ax2) = plt.subplots(2, 1)`.

3. Agreguen `sharex=True` y luego `sharey=True`. ¿Qué cambia? ¿Cuándo conviene cada uno?
4. Prueben `layout='constrained'` dentro de `plt.subplots`.
5. Hagan una cuadrícula de 2×2 con los niveles 2, 10, 20 y 40. Llénenla con un ciclo:

```
fig, axs = plt.subplots(2, 2, figsize=(10, 6))
for ax, k in zip(axs.flat, [2, 10, 20, 40]):
    ax.plot(lat, ds['t'].sel(lon=100, lev=k))
    ax.set_title(f'Nivel {k}')
fig.suptitle('Temperatura en la columna lon=100')
```

c) GridSpec: gráficas de distinto tamaño.

```
fig = plt.figure(figsize=(10, 7))
gs = fig.add_gridspec(2, 2)
ax1 = fig.add_subplot(gs[0, :])    # fila 0, todas las columnas
ax2 = fig.add_subplot(gs[1, 0])
ax3 = fig.add_subplot(gs[1, 1])
```

1. Pongan `t_2` y `t_10` juntas en `ax1`, `t_2` en `ax2` y `t_10` en `ax3`.
2. Cambien a `add_gridspec(3, 2)` y hagan que `ax1` ocupe dos filas.

d) `add_axes`: la posición a mano.

```
fig = plt.figure(figsize=(10, 7))
ax1 = fig.add_axes([0.10, 0.55, 0.80, 0.35])   # [izq, abajo, ancho, alto]
ax2 = fig.add_axes([0.15, 0.10, 0.30, 0.30])
ax3 = fig.add_axes([0.55, 0.10, 0.30, 0.30])
```

1. Dibujen lo mismo que en GridSpec.
2. Hagan un eje pequeño *dentro* de `ax1`. ¿Se podría con `subplots`?

7. Dos ejes y en una gráfica

1. `ax2 = ax1.twinx()` crea un segundo eje y a la derecha, con su propia escala. Sirve para dos cantidades con unidades distintas. Elijan una pareja, en la columna `lon=100`:
   - temperatura y humedad (`sphum`) en `lev=50`, cerca de la superficie;
   - temperatura y ozono (`o3mr`) en `lev=10`;
   - temperatura y altura del nivel 10 (`zh`, como en la sección 2).

```
# ejemplo con la primera pareja
t_50 = ds['t'].sel(lon=100, lev=50)
h_50 = ds['sphum'].sel(lon=100, lev=50)

fig, ax1 = plt.subplots(figsize=(8, 4))
ax1.plot(lat, t_50, color='crimson')
ax2 = ax1.twinx()
ax2.plot(lat, h_50, color='steelblue')
```

2. Pongan nombre y unidades a los dos ejes y.
3. Coloreen cada eje como su línea: `ax1.tick_params(axis='y', colors='crimson')`.
4. ¿Por qué no tiene sentido usar `twinx` con `t_2` y `t_10`?
5. `secondary_yaxis` agrega un eje que es la misma escala convertida. Pongan uno en °C:

```
ax.secondary_yaxis('right', functions=(lambda k: k - 273.15,
                                       lambda c: c + 273.15))
```

6. Pongan mal la segunda función a propósito. ¿Qué pasa?
7. En una frase cada uno: ¿en qué se diferencian `twinx` y `secondary_yaxis`?

8. Perfil vertical: ejes invertidos y logarítmicos

Ahora fijamos un punto (latitud y longitud) y dejamos libre el nivel. Quitamos `lev=0`:

```
col = dict(lat=100, lon=100)
t_col = ds['t'].sel(**col).values[1:]

zh = ds['zh'].sel(**col).values
z_km = 0.5 * (zh[:-1] + zh[1:])[1:] / 1000        # altura a media capa

delp = ds['delp'].sel(**col)
p_hpa = (delp.cumsum('lev') - delp / 2).values[1:] / 100   # presión a media capa
```

1. Grafiquen la temperatura en x y la altura en y.
2. Cambien la altura por la presión. Queda de cabeza: arréglenlo con `ax.invert_yaxis()`.
3. Pongan la presión en escala logarítmica (`ax.set_yscale('log')`) con marcas en 1000, 500, 200, 100, 50, 10 y 1 hPa (`ax.set_yticks(valores, etiquetas)`).
4. Marquen un nivel con `ax.axhline(...)` y pónganle `label`.
5. Con un ciclo, superpongan los perfiles de 5 puntos distintos con `alpha`.

9. Sombrear regiones

1. Grafiquen `t_2` y `t_10` contra `lat` y sombreen entre ellas: `ax.fill_between(lat, t_2, t_10, alpha=0.3)`.
2. Sombreen solo donde `t_2` pase de un valor, con `where=t_2 > valor`.
3. Calculen la anomalía `anom = t_10 - t_10.mean()`. Sombreen de un color donde es positiva y de otro donde es negativa, contra el cero. Hagan una versión sin y otra con `interpolate=True`. ¿Qué cambia en los cruces por cero?
4. Calculen para cada nivel el promedio, el mínimo y el máximo de `t` en todo el dominio (`.mean(['lat', 'lon'])`, etc.). Grafiquen el promedio como perfil y la banda mínimo–máximo con `ax.fill_betweenx`.
5. Hagan que la banda aparezca en la leyenda.

10. Histogramas

Usen todos los puntos de un nivel, aplanados: `ds['t'].sel(lev=10).values.ravel()`.

1. Histograma de `t` en `lev=10` con `ax.hist`. Prueben `bins=` 10, 50 y 200.
2. Agreguen `density=True`. ¿Qué representa ahora el eje y?
3. Superpongan el histograma de otro nivel, con los mismos bordes para los dos (`np.linspace`). Usen `histtype='step'` o `alpha`, y leyenda.
4. *(Opcional)* Histograma acumulado con `cumulative=True`. Marquen la mediana con `ax.axhline(0.5)`.

11. Dispersión

Usen `lev=50`, cerca de la superficie: arriba casi no hay vapor de agua.

1. Hagan `ax.scatter` de `t` contra `sphum`. Prueben `s=` y `alpha=`. ¿Qué problema tiene con tantos puntos?
2. Coloreen por latitud (`c=ds['geolat'].values.ravel()`, `cmap=`) y agreguen `fig.colorbar`.
3. Repítanlo con `ax.hist2d` y con `ax.hexbin`. Prueben `bins` y `gridsize`.
4. Quiten las celdas vacías (`cmin=1` / `mincnt=1`) y usen escala logarítmica (`norm=LogNorm()` / `bins='log'`). `LogNorm` se importa con `from matplotlib.colors import LogNorm`. Agreguen barra de color a los dos.
5. *(Opcional)* Scatter de `o3mr` contra `t` en `lev=10`, coloreado por latitud.

12. Textos y anotaciones

Sobre `t_10` contra `lat`:

1. Escriban un texto con `ax.text(x, y, '...')` en coordenadas de los datos, y otro fijo en la esquina superior izquierda con `transform=ax.transAxes`.
2. Encuentren el máximo de `t_10` (`argmax`) y señálenlo con `ax.annotate`, con flecha y el valor.
3. Cambien las marcas del eje x a `[0, 10, 20, 30, 40]` con etiquetas tipo `'10°N'`, y rótenlas: `ax.tick_params(axis='x', rotation=45)`.
4. Sombreen la franja tropical (hasta 23.5°) con `ax.axvspan`.
5. Usen LaTeX en las etiquetas: `r'$T$ [K]'`.